

Python Conversational Assistant Using Intent Rules

GenAI Course – Practical Assessment

Student: P Venkata Harshavardhan Reddy

Register Number: 99230040739

Department: CSE

Programming Language: Python

1. Project Overview

This assessment implements a lightweight conversational assistant in Python without external AI APIs. Instead of attempting to understand every possible sentence, the program maps recognizable phrases to a small set of intents such as greeting, capabilities, study guidance, and termination. The design emphasizes readable Python logic, input normalization, and predictable responses.

2. Functional Requirements

Requirement	Implementation
User identification	Collect a display name before the conversation begins.
Input cleanup	Trim whitespace and convert text to lowercase.
Intent recognition	Check phrases against ordered keyword groups.
Useful responses	Provide responses for greeting, capabilities and Python-study guidance.
Unknown input	Return a neutral fallback message and suggest available commands.
Termination	Stop when the user enters stop, quit, or bye.

3. Python Implementation

```
def study_assistant():
    student = input("Enter your name: ").strip()
    print(f"Assistant: Welcome, {student}!")

    while True:
        message = input("You: ").strip().lower()

        if message in ("stop", "quit", "bye"):
            print("Assistant: Session closed. Keep learning!")
            break

        if any(word in message for word in ("hello", "hi", "hey")):
            print("Assistant: Hello! What would you like to learn?")
        elif "python" in message and any(word in message for word in ("learn", "roadmap", "study")):
            print("Assistant: Start with syntax, conditions, loops, functions, collections,")
            print("then move to OOP, files, exceptions and small projects.")
        elif any(word in message for word in ("feature", "help", "do")):
            print("Assistant: I can greet you, suggest a Python learning path, or end the session.")
        elif "name" in message:
            print("Assistant: I am a simple Python study assistant.")
        else:
            print("Assistant: I did not recognize that request.")
            print("Assistant: Try asking for help, Python study guidance, or say bye.")

if __name__ == "__main__":
    study_assistant()
```

4. Logic Explanation

The application begins by collecting a name and then enters a continuous interaction loop. Every message is cleaned before matching. Exit commands are checked first so that the session can terminate immediately. The remaining conditions classify the message using keyword groups. Because the rules are evaluated from top to bottom, the order of conditions controls which response is selected. Inputs outside the supported intents receive a fallback response.

5. Test Cases

Input	Expected behavior
hello	Displays a greeting.
give me a Python study roadmap	Returns a compact Python learning sequence.
what can you do?	Lists the assistant's supported functions.
random question	Uses the fallback response.
bye	Prints a closing message and ends the loop.

6. Learning Outcome

The exercise demonstrates how Python strings, loops, conditional branching, functions, formatted output, and basic intent rules can be combined to build a deterministic conversational interface. It also illustrates why input normalization is important when user messages can vary in capitalization and spacing.